



Technical Terms Glossary

61 terms from the project and the study series, defined plainly and grounded in the technical report — the shared vocabulary the scenarios and viva answers rely on.

Project: OptiDBX — Safe PostgreSQL Auto-Tuning

Document: 3 of the OptiDBX Study Series

Grounded in: docs/optidbx_technical_report.md

Prepared: 22 September 2026

THE OPTIDBX STUDY SERIES — READ THESE TOGETHER

1 Demo Scenarios Study Guide

Understand every scenario the dashboard can demonstrate.

2 Viva Question Bank

Questions a panel may ask, with grounded answers.

3 Technical Terms Glossary

Every term in the project, defined plainly.

YOU ARE HERE

5 Page-by-Page Presentation Scripts

What to say on each page of the dashboard.

HOW THIS GLOSSARY IS ORGANISED

Technical Terms Glossary

Every term used across OptiDBX and the other study documents, A–Z. Each entry has a plain definition, a category tag, and — where useful — a [See](#) link to the matching scenario (Doc 1), viva question (Doc 2), or technical-report section (e.g. §4.3).

MEASUREMENT

DECISION

SAFETY

ARCHITECTURE

DETECTION

POSTGRESQL

INFRASTRUCTURE

RESULTS

Reading tip. If you learn six terms first, learn these: **owned workload**, **p95 / tail latency**, **warm-up exclusion**, **net-benefit rule**, **verified rollback**, and **fail-closed**. Almost every decision in the project is explained by those six.

Definitions

Sorted alphabetically; category tags let you scan by theme.

A

Approved values (whitelist)

DECISION

The fixed set **{1, 2, 4, 6, 8}** that `max_parallel_workers_per_gather` may take. The tuner only moves between adjacent approved values, one step at a time, so every change is bounded and reversible.

See: Doc 1 Scenario 4 · §3.3

Audit row

SAFETY

A persisted record written *before* a mutation describing the action. Part of journal-before-mutation; if it cannot be written, no change occurs.

See: §3.3

B

Baseline

MEASUREMENT

A warm, steady-state reading of the owned workload taken before any change — the ‘before’ in the KEEP/ROLLBACK comparison. Each action gets a **fresh** baseline; evidence never carries over between actions.

See: Doc 1 Scenario 4 · §4.2

Bounded observation

ARCHITECTURE

The fixed **30-second** window after a change during which the owned workload is measured. The first interval is excluded to avoid transition effects.

See: §3.3

Client-observed measurement

MEASUREMENT

Timing each query at the **client** around its round trip, instead of reading server counters — it captures what the user actually experiences.

See: §4.1

Closed monitoring loop

ARCHITECTURE

The full cycle: workload → OS+DB telemetry → detection → recommendation → (display | safe apply) → bounded observation → KEEP/ROLLBACK → cooldown → monitor.

See: §3

Cold-start artifact

RESULTS

The bias where a baseline taken on cold caches/connections understates baseline performance and **inflates the apparent improvement**. It once made a reduction look like a $\sim 2\times$ gain ($\approx 6.4 \rightarrow \approx 12$ QPS) versus a true ≈ 11 QPS. The reason warm-up exclusion exists.

See: Doc 2 Q7.1 · §6.2

Context switches

DETECTION

OS events where a CPU switches between threads/processes. A high rate signals CPU oversubscription (too many workers contending) and is one of the detection signals.

See: §3.2

Cooldown

ARCHITECTURE

A **30-second** pause after any KEEP or ROLLBACK before the tuner may act again, letting the system settle.

See: §3.3

Core pinning (CPU affinity)

INFRASTRUCTURE

Binding PostgreSQL (or the monitor) to a subset of CPU cores. Used both to *construct* oversubscription for the experiments and to *reserve* CPU so the monitor keeps cadence.

See: §6.5 · §6.6

CPU oversubscription

RESULTS

When more parallel workers demand CPU than there are cores to run them, so they contend. This is exactly the condition the reduction action targets.

See: Doc 1 Scenario 1 · §6.4

Debouncing

DETECTION

Requiring a signal to persist before acting. Here, three consecutive problematic readings are needed, so a single spike is ignored.

See: Doc 1 Scenario 7 · §3.2

Detection

ARCHITECTURE

The stage that confirms a bottleneck via three consecutive intervals, each meeting a conjunction of CPU, context-switch, worker-count and latency conditions.

See: §3.2

F

Fail-closed

SAFETY

A design where, on uncertainty (e.g. an unverifiable rollback), the system **stops and blocks further action** rather than risk an unsafe change.

See: Doc 1 Scenario 9 · §3.3

False positive

DETECTION

Acting on a bottleneck that is not real (e.g. a transient spike). The three-reading rule exists to prevent this.

See: Doc 1 Scenario 7

FastAPI

INFRASTRUCTURE

The Python web framework serving the tuner's backend API to the dashboard.

fsync

SAFETY

A system call that forces buffered data to durable storage. The recovery journal is fsync-ed *before* any mutation so it survives a crash.

See: §3.3

G

Gather node

POSTGRESQL

The query-plan node that collects rows from parallel workers. `max_parallel_workers_per_gather` caps how many workers a single Gather may use.

Ground-truth sweep

RESULTS

Measuring performance at fixed parallelism with the tuner **off**, to establish whether a reduction *can* help at all — independent of the tuner's decisions.

See: §6.1 · §6.4

H

Hard guard / cap / veto

DECISION

Overrides in the net-benefit rule: never keep if tail latency regresses beyond ~**10%**, throughput regresses beyond its cap, error rate rises, or OS resources degrade.

See: §4.3

J

Journal-before-mutation

SAFETY

Writing and fsync-ing the recovery record plus audit row **before** changing anything. If either write fails, no mutation happens — the durable journal is always ahead of the live change.

See: §3.3

K

KEEP

DECISION

The verdict that a change measurably helped — it cleared the net threshold with no regression — and is retained.

See: Doc 1 Scenarios 1–4 · §4.3

L

latency_first / throughput_first

DECISION

Alternative single-axis decision policies, provided only for comparison. The default is `net_benefit`.

See: §4.3

M

max_parallel_workers_per_gather

POSTGRESQL

The single knob OptiDBX tunes: the maximum parallel workers a single Gather node may use. Also called **per-gather parallelism**.

See: §3.3

Median latency (p50)

MEASUREMENT

The middle query latency, reported alongside p95. Less sensitive to the slow tail than p95.

See: §4.1

Memory-bandwidth bound

RESULTS

When performance is limited by RAM throughput rather than CPU. On the well-provisioned host the workload was memory-bandwidth bound, so changing worker count did not help.

See: Doc 1 Scenario 6 · §6.1

Monitor starvation

RESULTS

When near-100% CPU starves the monitoring loop so it cannot hold its 5 s cadence, causing valid actions to be rejected. Reserving CPU for the monitor fixes it.

See: Doc 2 Q7.3 · §6.6

N

Net threshold

DECISION

The **5%** minimum net improvement a change must clear to be kept.

See: §5

net-benefit rule (net_benefit)

DECISION

The default KEEP rule: keep when a weighted combination of throughput gain and latency improvement exceeds a **5% net threshold**, subject to the hard guards.

See: Doc 2 Q4.5 · §4.3

NO_ACTION

DECISION

The verdict that no sustained contention was confirmed, so nothing is changed — a first-class *correct* outcome on a healthy host.

See: Doc 1 Scenarios 6 & 7 · §7



Offline tuning

ARCHITECTURE

Exploring configurations on a replica or session and then installing one. Contrast **online tuning**.

See: §1

Online tuning

ARCHITECTURE

Observing a live workload, applying a change, and judging its effect immediately — OptiDBX's setting, and the reason the safety and measurement layer is hard.

See: §1

Owned workload

MEASUREMENT

The queries OptiDBX itself runs on explicitly bound sessions. It is the **only** workload measured for decisions and the only sessions the tuner may change.

See: Doc 2 Q4.1 · §4.1



p95 latency (tail latency)

MEASUREMENT

The 95th-percentile query latency — the slow tail users feel. The **decisive** signal, because reducing parallelism mainly improves the tail while throughput stays flat.

See: Doc 2 Q4.2 · §4.3

Parallel sequential scan

POSTGRESQL

A table scan split across parallel workers. It dominated the well-provisioned workload (~119 ms over cached data), making that workload memory-bandwidth bound.

See: §6.1

Parallel worker

POSTGRESQL

A background process executing part of a parallel query. Too many under CPU pressure cause contention.

pg_stat_database

POSTGRESQL

A server view of database-wide counters. Unsuitable as a *decision* signal because it counts all transactions (including the monitor's own) and misses a few large queries.

See: §4.1

pg_stat_statements

POSTGRESQL

A server view of per-statement aggregate statistics. Too coarse across statement types to decide on.

See: §4.1

pgbench

INFRASTRUCTURE

PostgreSQL's built-in benchmarking tool. Used here at **scale factor 10** (~1M rows, ≈150 MB) to build the dataset.

See: §5



React + Vite

INFRASTRUCTURE

The dashboard's frontend library (React) and build tool (Vite).

RECOMMENDATION (verdict)

DECISION

Surfacing a real condition that is **outside the safe action set** (e.g. memory pressure) to an operator, with evidence, instead of acting on it.

See: Doc 1 Scenario 8

Recommendation mode

ARCHITECTURE

A mode where the tuner detects and **recommends** but does not auto-apply. Live sessions start here.

Recovery journal

SAFETY

The fsync-ed on-disk record (target, before-state, scope) that stays authoritative after a crash and cannot be impersonated by a new connection.

See: Doc 2 Q5.3 · §3.3

Regression guard

DECISION

See **hard guard**: the cap that forbids keeping a change if tail latency (or throughput) worsens beyond a limit.

See: Doc 1 Scenario 5 · §4.3

Rejection memory

SAFETY

Recording performance rollbacks against a coarse workload-condition signature and suppressing the same reduction under comparable conditions until a TTL elapses.

See: §3.4

ROLLBACK

DECISION

The verdict that a change did not help (or hurt), so the original value is restored and verified.

See: Doc 1 Scenario 5 · §4.3

ROLLBACK_FAILED

SAFETY

The fail-closed state entered when a rollback cannot be verified: original sessions are retained and further actions blocked until an operator intervenes.

See: Doc 1 Scenario 9 · §3.3

S

Sampling interval

ARCHITECTURE

The fixed **5-second** cadence at which OS and DB telemetry are collected.

See: §3.1

Scale factor

INFRASTRUCTURE

pgbench's dataset-size multiplier; **10** here (~1M pgbench_accounts rows).

See: §5

Stale-timestamp defect

RESULTS

The bug where an auto-approved recommendation kept its first-detection timestamp while the baseline advanced, so the continuity check rejected it every interval. Fixed by refreshing the timestamp at approval. Explains the prior inability to ever KEEP.

See: Doc 2 Q7.2 · §6.3

T

Telemetry

ARCHITECTURE

The OS and DB metrics sampled each interval: CPU, memory, disk I/O, context switches; query latency, throughput, temp-file usage, active parallel workers.

See: §3.1

Temp-file spill

POSTGRESQL

When a query's working data exceeds memory and spills to temporary files on disk — a symptom of memory pressure.

See: Doc 1 Scenario 8

Three-reading confirmation

DETECTION

The rule that a bottleneck must persist across **three consecutive** intervals (~15 s) before any action.

See: Doc 1 Scenario 7 · §3.2

Throughput (QPS)

MEASUREMENT

Queries completed per second on the owned workload — one of the two decision axes (with p95 latency). QPS = queries per second.

See: §4.1

Timestamp continuity check

ARCHITECTURE

An apply-time check that a recommendation's baseline is continuous with the current interval. It misfired in the stale-timestamp defect.

See: §6.3

TTL (time-to-live)

SAFETY

The expiry after which a suppressed reduction may be reconsidered in rejection memory.

See: §3.4

V

Verified apply

SAFETY

Setting the new value and then **reading it back** to confirm the change actually took effect.

See: §3.3

Verified rollback

SAFETY

Restoring the original value and **reading it back** to confirm. Only a confirmed read-back counts as a successful rollback.

See: Doc 1 Scenario 5 · §3.3

W

Warm-up exclusion

MEASUREMENT

Requiring the workload to be warm and discarding an initial slice before the baseline and observation, to avoid cold-start inflation.

See: Doc 2 Q4.3 · §4.2

Workload-condition signature

SAFETY

The coarse fingerprint (parallel-worker band, CPU band) rejection memory uses to match ‘comparable conditions’.

See: §3.4

WSL2

INFRASTRUCTURE

Windows Subsystem for Linux v2 — the Ubuntu environment where PostgreSQL 16 runs under Windows 11.

See: §5